REVIEW 1: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 1: Introduction

1.1 Project Overview

The evolution of Command Line Interfaces (CLIs)

Command Line Interfaces (CLIs) have served as the foundational pillar of computing and system administration since the dawn of modern operating systems, originating from early teleprinter terminals, the UNIX shell (sh), and modern POSIX environments such as Bash and Zsh. CLIs offer unmatched speed, deterministic scripting capabilities, and direct low-level control over kernel resources, processes, and storage subsystems. However, the operational power of traditional CLIs is tied to an explicit syntax model: users must memorize exact command names, nuanced flags, and order-dependent arguments.

The shift toward Natural Language Processing (NLP) in system administration

Recent advancements in Natural Language Processing (NLP) and Large Language Models (LLMs) have sparked a paradigm shift in human-computer interaction. Instead of burdening system operators and developers with memorizing complex argument vectors, modern system tooling is moving toward 'Agentic Shells'—intelligent agents capable of understanding high-level human intent and translating natural language into safe, verifiable system calls. Within this ecosystem, the Smart Archive Agent serves as a specialized, autonomous specialist dedicated to the storage and archiving domain, providing safe file packaging, compression, decompression, integrity auditing, and cryptographic protection on Ubuntu Linux.

1.2 Problem Statement

Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Traditional archiving and compression tools in Linux (e.g., tar, gzip, bzip2, xz, zip, and gpg) exhibit distinct, often inconsistent syntax conventions. For example, creating an xz-compressed tarball requires remembering 'tar -cJf archive.tar.xz folder', whereas gzip requires '-czf' and bzip2 requires '-cjf'. In contrast, file-level compression utilities like xz or gzip compress in-place and delete the original file unless explicit piping flags ('-c') are passed. Furthermore, symmetric encryption with GPG introduces

additional parameters ('--symmetric', '--cipher-algo AES256'). Users who infrequently handle these tasks face significant friction and cognitive load.

Risk of executing incorrect commands due to human error

Executing erroneous archive commands in shell environments carries acute operational risks. A common mistake is argument inversion (e.g., swapping source and destination paths in tar), which can completely overwrite critical source directories with empty archives. Similarly, unvetted extraction commands can trigger path traversal vulnerabilities (Zip Slip) or overwrite existing production files. The Smart Archive Agent addresses these risks by providing a safe translation layer that dispatches only into bounded, pre-validated functions with explicit path verification and strict non-overwrite rules.

1.3 Objectives

- **Lightweight NL-to-CLI Translation:** To build a responsive, lightweight natural-language interpretation engine that parses natural language instructions into structured operations without requiring heavy third-party runtime dependencies.
- **Predefined Operation Boundaries:** To enforce safety by mapping requests strictly to predefined tool functions (create, extract, list, inspect, compress, decompress, encrypt, decrypt) rather than executing arbitrary shell command strings.
- **Model Context Protocol (MCP) Architectural Alignment:** To design tool definitions, input schemas, and execution workflows in alignment with the Model Context Protocol (MCP) standards, allowing seamless future integration with broader agent networks.
- **Security & Guardrail Integration:** To implement robust path validation, prevent Zip Slip directory traversal attacks, prevent accidental file overwrites, and mandate AES-256 GPG encryption for sensitive storage.
- **Structured Feedback & Auditing:** To return actionable success messages, output artifacts, entry listings, and descriptive error diagnostics directly to the user.

1.4 Scope and Limitations

- **Supported Operating Systems:** The Smart Archive Agent is natively targeted for Ubuntu Linux and Windows Subsystem for Linux (WSL 2). It leverages standard Linux binaries (tar, gzip, bzip2, xz, file, gpg, zip, unzip) and Python 3.14.
- **Terminal Environments:** Operates within standard terminal shells (Bash, Zsh) as an interactive REPL or direct CLI command tool.
- **Administrative Privileges:** The agent runs strictly with ordinary user privileges. It explicitly avoids requiring 'sudo' or root escalation, eliminating the possibility of root-level filesystem damage.
- **Domain Boundaries:** The agent is strictly bounded to file packaging, compression, and encryption. Unrelated queries (e.g., process management, network configuration, general Q&A) are identified and safely rejected.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs in structured text and code synthesis

Recent frontier models (such as GPT-4, Claude 3.5, and Gemini Pro) have demonstrated remarkable capabilities in translating unstructured natural language into structured outputs, API payloads, and executable code. Function calling and structured tool outputs enable language models to extract entities from natural text and populate predefined JSON schemas with high fidelity. In system administration, this capability enables flexible intent detection across diverse phrasings and languages.

Challenges with LLM hallucination in system-level execution

Despite their linguistic proficiency, LLMs suffer from probabilistic hallucinations—generating non-existent file paths, fabricating invalid command-line flags, or selecting destructive operations. In operating systems, executing an unvetted LLM-generated shell command directly in 'sh' or 'bash' is fundamentally insecure. To mitigate this hazard, our project adopts a strict 'Separation of Concerns': intent interpretation is kept separate from execution. The agent utilizes an intent router with schema-enforced tool calls, ensuring that every argument is verified by deterministic validation guards before reaching the OS.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP (Clients, Servers, and Tools)

The Model Context Protocol (MCP), open-sourced by Anthropic, is an open standard designed to connect AI applications with external tools, data sources, and execution environments. The MCP architecture comprises three key entities: (1) MCP Clients (the host application or agent orchestrator), (2) MCP Servers (lightweight programs exposing specialized capabilities), and (3) MCP Tools (individual functions declared with JSON Schema parameter specifications). MCP establishes clean contracts for discovery, input validation, and execution.

Why MCP is preferred over traditional rigid API tooling for local system access

Traditional system scripting interfaces rely either on brittle text-based CLI parsing or bespoke, proprietary RPC adapters. MCP provides standardized discovery, dynamic capability negotiation, and uniform error reporting. By adopting MCP-aligned tool interfaces, the Smart Archive Agent can run as a standalone terminal specialist today and seamlessly plug into an enterprise-wide multi-agent shell tomorrow without altering its core archiving and security implementations.

2.3 Existing Solutions

A comparative study of current commercial and open-source terminal assistance solutions reveals distinct architectural philosophies:

Solution	Primary Paradigm	Security / Sandboxing	Relevance to this Project

Traditional Bash	Manual command-line execution	User permissions only; no syntax safety or guardrails.	Reliable system backends, but requires exhaustive syntax memorization.
Warp AI	AI-assisted terminal emulator	Suggests shell commands; user must review before executing in terminal.	Demonstrates natural-language interaction in terminals, but executes arbitrary shell scripts.
GitHub Copilot CLI	NL-to-shell command synthesizer	Explains generated bash command; requires manual user confirmation.	Illustrates LLM-driven command generation; vulnerable to flag hallucinations.
Smart Archive Agent (Ours)	Domain-bounded Agentic Specialist	Strict function dispatch; path validation; Zip-Slip prevention; non-root execution.	Combines natural language ease with deterministic guardrails in a specialized storage domain.

References

[1] Anthropic, 'Model Context Protocol (MCP) Specification,' 2024. [Online]. Available: https://github.com/modelcontextprotocol/docs

[2] OpenAI, 'Function Calling and Structured Outputs Guide,' 2024. [Online]. Available: https://developers.openai.com/api/docs/guides/function-calling

[3] Free Software Foundation, 'GNU Tar Manual: Tape Archiving Utility,' GNU Project, 2023. [Online]. Available: https://www.gnu.org/software/tar/manual/

[4] Warp Technologies, 'Warp: The AI-Powered Terminal User Guide,' 2024. [Online]. Available: https://docs.warp.dev/

[5] GitHub Inc., 'GitHub Copilot in the CLI Documentation,' 2024. [Online]. Available: https://docs.github.com/en/copilot/using-github-copilot-in-the-cli